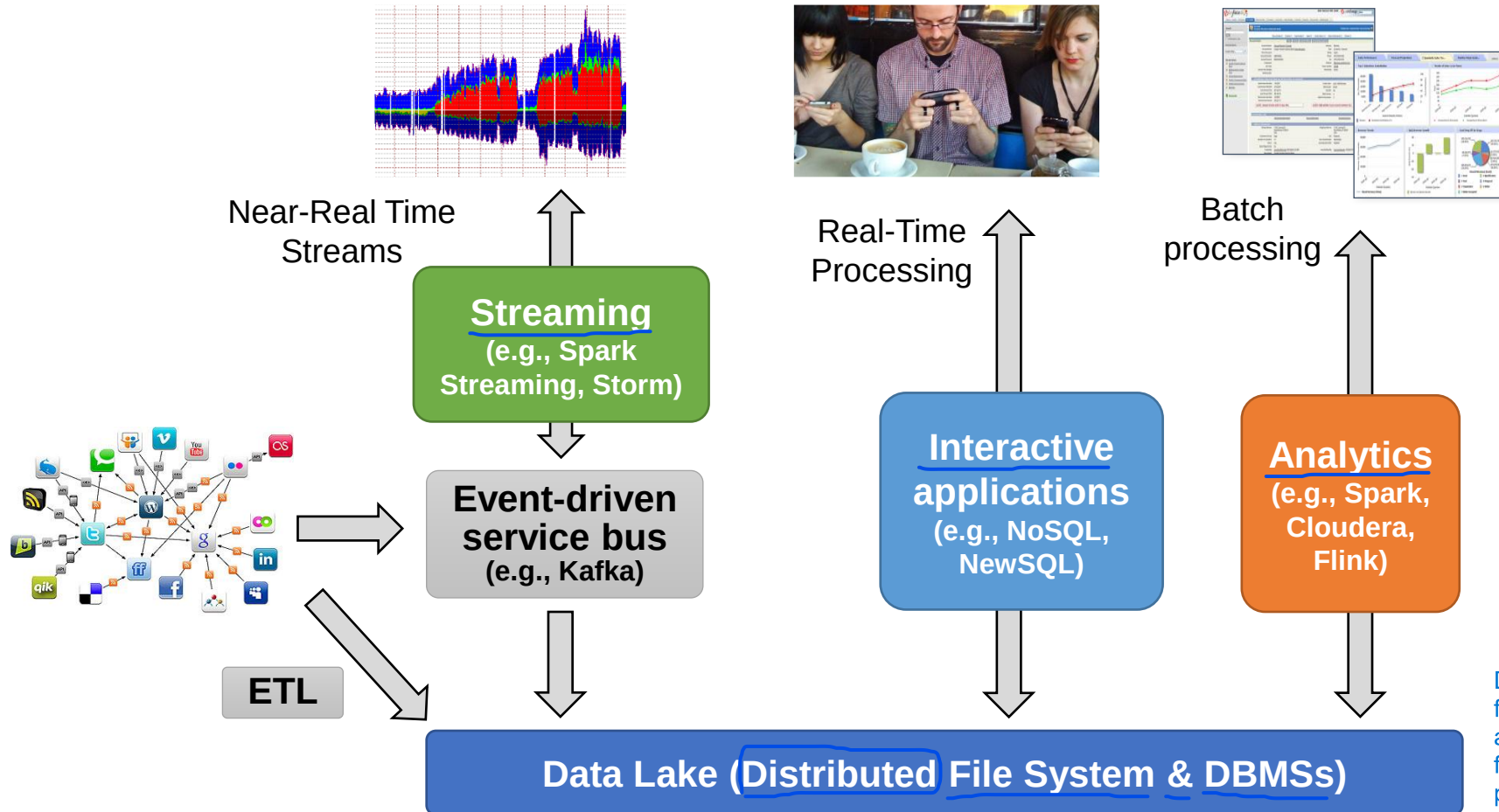


STORAGE:

Distributed File System

(H)DFS, File formats,
Data lake and lakehouse

The Big Data flow



Distributed File System is fundamental for Interactive and Batch processing, is not fundamental for Streaming processing (but in most cases you will save the data even in Streaming processing)

(H)DFS

Quando sappiamo come funziona il DFS di Hadoop (open-source), sappiamo bene o male gestire anche quelli proprietari, che sono delle estensioni di quello open-source

(Hadoop) Distributed File System

Goals of a DFS

Big data support

- Requirement: files may be too big to be processed by a single machine (or even bigger than single disks)

Abstraction

- Requirement: users should interact with disks on different machines as if they were one, unified disk

Fault-tolerance

- Requirement: disks can fail, machines can be unreachable, but data should always be available
- "i problemi sono trasparenti all'utente": l'intero processo di gestione e mascheramento dei guasti avviene completamente dietro le quinte, senza che l'utente o l'applicazione client debbano preoccuparsene o subire interruzioni

L'espressione 'streaming data access patterns' non ha nulla a che fare con "streaming analysis". Significa semplicemente che la modalità tipica di interazione con i file in HDFS consiste nel leggere sequenzialmente ed in modo continuo ampie porzioni di dati (se non l'intero file). L'architettura di HDFS è progettata per massimizzare il throughput (volume di dati trasferiti nell'unità di tempo) a scapito della latenza (tempo di risposta per accedere a un singolo dato casuale).

HDFS

Nel mondo Big Data il termine Streaming viene usato con due significati molto diversi:

- Streaming Analytics / Real-Time Processing: Si riferisce all'elaborazione continua di flussi improvvisi di eventi in tempo reale (es. eventi Kafka, Spark Streaming, Flink).
- Streaming Data Access (in HDFS): Si riferisce alla modalità meccanica con cui un'applicazione legge un file dal disco system. Significa che il file viene aperto e "srotolato" (letto in modo sequenziale e ininterrotto come un nastro/flusso) dall'inizio alla fine o per ampi blocchi continui.

- Throughput (Capacità di trasmissione): Misura quanti dati il sistema riesce a leggere o scrivere nell'unità di tempo (es. Gigabyte al secondo). HDFS è ottimizzato al massimo per questo scopo: muovere grandi volumi di dati verso i motori di calcolo nel minor tempo complessivo possibile.

- Latenza (Tempo di risposta): Misura quanto tempo impiega il sistema per rispondere a una singola richiesta specifica (es. recuperare una riga di dati entro pochi millisecondi). HDFS non è adatto a query interattive ad altissima velocità o ad accessi punto-a-punto a bassa latenza; per quel tipo di workload si utilizzano invece i database NoSQL o i database relazionali tradizionali.

HDFS is a filesystem designed for storing **very large files** with **streaming data access** patterns, running on clusters of **commodity hardware**

- A typical file in HDFS is **gigabytes to terabytes in size**. There are Hadoop clusters running today that store petabytes of data
- **Applications that run on HDFS need streaming access to their data sets.** HDFS is designed more for batch processing rather than interactive use. The emphasis is on high throughput of data access rather than low latency of data access
- **HDFS applications need a write-once-read-many access model for files.** A file once created, written, and closed need not be changed. This assumption simplifies data coherency issues and enables high throughput data access
- **Hardware failure is the norm rather than the exception.** Therefore, detection of faults and quick, automatic recovery from them is a core architectural goal of HDFS

HDFS non è progettato per l'aggiornamento o la modifica diretta dei dati (in-place updates). Il suo funzionamento si basa sul modello Write-Once, Read-Many (WORM): un file viene creato, scritto e chiuso, senza la possibilità di cambiarne i singoli blocchi o record già salvati. L'unica operazione di scrittura successiva consentita è l'aggiunta di nuovi dati in coda (append); qualsiasi modifica a dati già esistenti richiede la riscrittura completa dell'intero file. HDFS si focalizza sull'archiviazione distribuita a basso costo e sull'accesso sequenziale ad alto throughput, delegando la gestione delle transazioni e delle modifiche puntuali ai DBMS.

HDFS: blocks

Why blocks?

- Files can be larger than disks.
- By splitting files into blocks, storage management is simplified and replication is easier

The block is the minimum amount of data read/written

(Blocks are small enough to be easily handled by a machine and large enough to contain a good quantity of data that can be sent with a high throughput)

- Disk blocks are normally 512B
- Filesystem blocks are typically a few KB
- HDFS blocks range between 64MB and 1GB (default: 128MB)
 - If a file is smaller, it will occupy the necessary disk space, not the full block size

→ Sebbene la dimensione logica del blocco sia grande (es. 128 MB), un file di dimensioni inferiori (es. 10 MB) occuperà su disco solo lo spazio effettivo necessario (10 MB) e non l'intera dimensione del blocco.

Why this big?

- Large files split into many small blocks require a huge number of seeks
- E.g.: block size = 4KB; file = 1GB; number of seeks = 250.000

(this will not be very optimised: troppo tempo per inviare il file)

→ tempi di ricerca del disco

Why not bigger?

HDFS: namenodes and datanodes

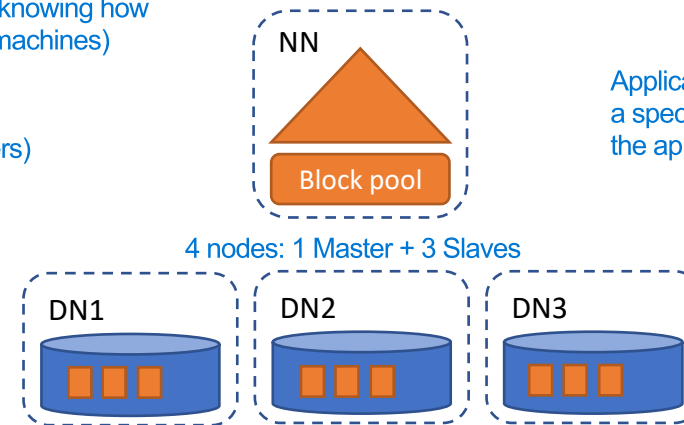
Nodes in an HDFS cluster operate in a master-slave pattern

Namenode (NN) - the master (the Service that is in charge of knowing how the data is distributed on other machines)

- Persistently maintains the filesystem tree and all files' and directories' metadata (organisation of folders)
- Keeps **in memory** the location of each block for a given file (*block pool*) (for each file, how many block and where are stored)

Datanodes (DNs) - the slave

- the Service that Store and retrieve blocks
- Periodically report to the NN with the list of blocks they are storing; *heartbeats* are sent to the NN to signal their active state (e.g., every 10 minutes)



HDFS: the SPoF

Il NameNode (NN) rappresenta il Single Point of Failure nell'architettura HDFS: gestendo l'albero del file system e la mappatura dei blocchi sui DataNode, la sua caduta rende l'intero cluster inaccessibile, anche se i dati fisici sui DataNode rimangono intatti.

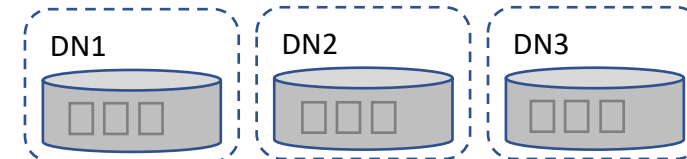
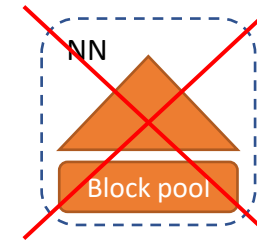
The NN is a single point of failure: without it, the filesystem cannot be used (no way to reconstruct the files from the blocks in the DN's)

1 **Backup solution** backup of the machine that runs the namenode (most basic solution, not the best one: it takes a lot of time to restart the NameNode)

- NN writes its persistent state to multiple filesystems, preventing loss of data

2 **Secondary NN solution** Il Secondary NameNode non è un NameNode di riserva pronto a subentrare in automatico.

- A separate machine regularly connects with the primary NN to build snapshots of its persistent data and saves them to local or remote directories
- These *checkpoints* can be used to restart a failed NN without having to replay the entire journal of file-system actions
- Still, restarting a NN could take 30+ minutes →



Sebbene il Secondary NameNode prevenga la perdita di dati, non offre disponibilità continua (High Availability): In caso di crash del NameNode primario, il passaggio o il riavvio non è istantaneo. La ricostruzione dello stato e il caricamento in memoria della mappa dei blocchi tramite l'ultimo checkpoint richiedono un intervento manuale o uno scatto di procedura che può impiegare 30 minuti o più, durante i quali il cluster rimane totalmente inattivo.

3 HDFS: High-Availability

High-Availability (HA) indicates a system that can tolerate faults

- The service never stops while the fault (be it hardware or software) is detected, reported, masked, and repaired off-line (errors are transparent to the users: system is quite-always available even if of failures)
- Most big data tools offer HA (not just HDFS)

Similar to the Secondary NN solution

HA is supported by configuring two separate machines as NNs

- One is in an *active* state, the other is in a *standby* state (the second one is continuously active (always synced with the first NN)). You can have 2+ NNs to have more redundancy and fault-tolerance
- In the event of a failure of the active NN, the standby NN takes over

The NN in standby keeps its metadata^{and the file system tree} up-to-date by:

- Reading the *edit log* files written by the active NN in a shared storage
 - Edit logs are typically stored in *Journal Nodes*, which are also replicated
- Receiving the data block locations^{and the file system tree} and the heartbeats directly from the DNs, which are configured to report to both NNs

(prende posto del primo NN, se cade, in modo immediato)

(more services up)

A HA solution is more efficient in recovering^(not so relevant), but requires more resources^(more services up) and communications^(not so relevant)

It isn't a Fault tolerant solution for NNs

HDFS: federation

Federation means that you have multiple Masters (NNs) that handle different portion of the File system (namespaces)

The size of the block pool^{and file system tree} is limited by the memory^(RAM) size of the NN

- May incur in scaling issues on large clusters with many files

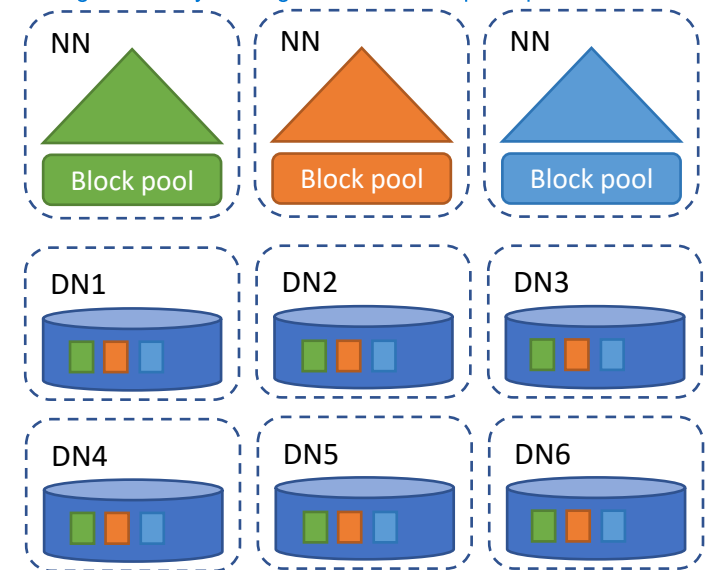
Solution: configure additional NNs

- Each NN manages a portion of the filesystem (*namespace*)
- NNs are independent of each other

Many advantages

- Performance** The Masters (NNs) are the ones that handle the communication with clients that ask for data access, so having multiple Masters can help handle multiple requests (helps even the division of the file system in namespaces)
 - NN can become a bottleneck for data access
- Availability**
 - In case of one NN's failure, the others are available
- Scalability**
 - By storing metadata in memory, the size of the NN's heap limits the number of files and blocks
- Maintainability, Security & Flexibility**
 - Each namespace is isolated and not aware of the others

Oltre alla federation, si può applicare, insieme ad esso, la soluzione High availability dove ogni NN ha la sua replica up-to-date



Cosa bisogna fare se i DataNodes falliscono? Quindi, bisogna rendere i dati disponibili (dati devono rimanere disponibili anche se il nodo che li gestisce va giù). La strategia tipica è quella di usare delle repliche dei blocchi

1 HDFS: replication

Each data block is independently replicated at multiple DNs in order to improve performance and robustness

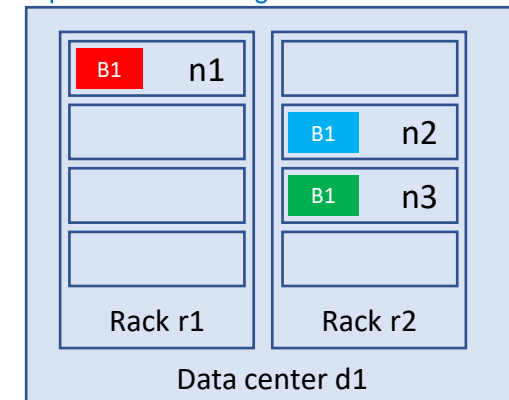
- Replication is aware of the cluster topology
- For each data block, the NN stores the list of DNs storing it

- Obiettivi della replicazione: Ogni blocco di dati viene salvato in copie multiple su DataNode distinti per garantire la sopravvivenza del dato in caso di rottura dei dischi o delle macchine, e per aumentare il parallelismo e le prestazioni delle letture distribuite.
- Consapevolezza della topologia (Rack Awareness): Il NameNode conosce la collocazione fisica dei DataNode all'interno dei singoli rack. Ciò consente di distribuire le repliche in modo strategico per sopravvivere ai guasti riguardanti al rack (es. la rottura di uno switch di rack), oltre a quello del nodo singolo.

The default replication factor is 3:

- Replica 1** is stored on the node (n1) where the client issued the write command (if the client resides within the cluster)
- Replica 2** is stored on a node (n2) in a rack (r2) different from the one of n1 (off-rack)
- Replica 3** is stored on a node, different from n2, but that belongs to r2

Contro: Significativo aumento dello spazio di archiviazione dovuto alla ridondanza dei dati.



Replicas can be rebalanced when nodes are added or unavailable

- Failover dei blocchi: Se un DataNode o un disco fallisce, il NameNode rileva la mancata ricezione degli heartbeat, nota che il fattore di replicazione è sceso sotto quota 3 e pianifica la copia del blocco mancante su un DataNode sano.

- Rebalancing del cluster: Quando vengono aggiunti nuovi DataNode vuoti, le repliche possono essere riequilibrate tra i nodi per evitare sbilanciamenti di memoria fisica o di I/O.

Un'altra strategia per il problema riguardante i failure dei DataNodes

2A HDFS: replication

L'Erasure Coding (EC) è una tecnica di Fault tolerance che sostituisce la duplicazione fisica dei blocchi con un calcolo matematico di parità.

HDFS supports Erasure Coding (EC) as an alternative to simple replication

- A mechanism similar to Redundant Array of Inexpensive Disks (RAID) 5-6
- Instead of replicating each block, blocks are striped
- A stripe is a sequence of chunks, composed of m data chunks + k parity chunks
- The data in a block may end up in multiple stripes

mechanism that you have on disks

i blocchi sono sottoposti a striping (i blocchi sono composti da più stripe)

di parità

possono essere distribuiti

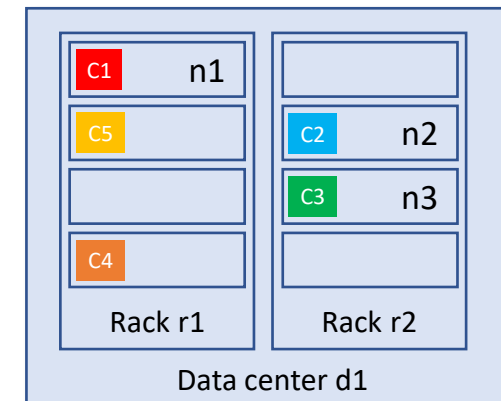
La quantità k viene decisa dagli amministratori di sistema in base al livello di sicurezza desiderato: Numero massimo di guasti tollerabili contemporaneamente = k
- Tolleranza ai guasti: Se si imposta $k = 3$, il sistema può tollerare la perdita o l'inaccessibilità simultanea di fino a 3 DataNode/dischi all'interno della stessa striscia senza perdere alcun dato.

I data chunk sono frammenti sequenziali costitutivi del dato originale. A ogni sequenza di m data chunk viene associato un set di k parity chunk: si tratta di blocchi di ridondanza calcolati algebricamente che garantiscono la tolleranza ai guasti, consentendo al sistema di ricostruire esattamente qualsiasi data chunk andato perso o danneggiato senza dover duplicare l'intero file.

Stripe

Chunk C1	1	0	0	1	1	1	0	1	0
Chunk C2	0	0	0	1	1	1	0	1	1
Chunk C3	1	0	1	0	1	0	1	0	0
Chunk C4	1	1	0	0	0	0	1	1	1
Parity C5	1	1	1	0	1	0	0	1	0

In questo caso, abbiamo 4 data chunks e 1 parity chunks ed i 5 DataNode salvano ognuno un data chunk o parity chunk



I chunk di uno Stripe sono distribuiti su rack differenti (es. C1 nel DataNode n1, C4, C5 nel Rack r1; C2, C3 nei DataNode n2 e n3 del Rack r2) per far sì che il sistema sopravviva anche al guasto di un intero rack.

Lo Stripe è l'unità logica orizzontale composta dall'insieme di tutti i m data chunk + i k parity chunk: Nell'immagine, lo Stripe è l'intera riga orizzontale che comprende C1+C2+C3+C4+C5. I dati di un data block possono estendersi su più Stripe successivi.

<https://docs.cloudera.com/runtime/7.3.1/scaling-namespaces/topics/hdfs-ec-overview.html>

C5 è il risultato di un'operazione logica (es. XOR / Reed-Solomon) applicata colonna per colonna sui bit di C1, C2, C3, C4. Se C2 va perso a causa del guasto di n2, il sistema combina i bit superstiti di C1, C3, C4 e C5 per ricalcolare e ripristinare esattamente la sequenza di C2.

Enrico Gallinucci – University of Bologna

BIG DATA

← additional data

In this case, 1 if odd, 0 if even

Non contiene dati dell'utente, ma un calcolo matematico effettuato sui dati di C1, C2, C3, C4

2_B HDFS: replication

Erasure Coding (EC) works best for cold datasets with relatively low I/O activities

EC è progettato principalmente per i cosiddetti cold datasets, ovvero volumi di dati storici o di archivio poco attivi, caratterizzati da basse frequenze di lettura e scrittura (basso I/O). Poiché EC scambia lo spazio su disco con un maggiore consumo di CPU e banda di rete, applicarlo su hot datasets (molto frequentati) penalizzerebbe le prestazioni del cluster.

Main advantages

Main reason for EC

- Sensibly reduce data redundancy (from 200% to 50% with default setups)

Rispetto alla prima soluzione, EC richiede meno spazio (viene replicata solo una parte del dato totale)

- If m parity chunks are created every k chunks, the redundancy factor is m/k

- The default is 3 parity chunks every 6 chunks

La configurazione Default di EC è 6 data chunk e 3 parity chunk per Stripe

- Faster writes (chunks are distributed)

Poiché i vari chunk che compongono la striscia vengono distribuiti ed emessi in parallelo verso molteplici DataNode fisici contemporaneamente, la velocità di scrittura sfrutta la larghezza di banda e l'I/O combinato di più dischi in parallelo.

Main disadvantages

- Higher CPU cost in both reading and writing

In Scrittura, il sistema deve eseguire algoritmi algebrici (Reed-Solomon) su matrici di bit per calcolare i chunk di parità prima dell'invio. In Lettura, la ricomposizione delle strisce richiede overhead computazionale da parte delle CPU dei nodi.

- Longer recovery time in case of failure

Con EC, il ripristino di un chunk perso richiede di scaricare via rete i restanti chunk dello stripe da tutti gli altri nodi e ri-calcolare algebricamente il dato mancante tramite CPU.

- Loss of data locality (to read a block, you need to get its stripes from multiple machines)

A differenza della prima soluzione di replicazione, con EC accedere ad un blocco richiede di recuperare i singoli chunk distribuiti su più macchine differenti per poter ricomporre il dato originale. Questo approccio comporta un overhead architetturale maggiore rispetto alla replicazione semplice: non è più possibile leggere un intero blocco in locale dallo stesso nodo di calcolo, costringendo il sistema ad accedere sempre alla rete. La ricomposizione dei chunk e l'eventuale decodifica algebrica dei dati richiedono più risorse computazionali e generano un traffico di I/O diffuso tra i nodi del cluster

Con EC, poiché un blocco è frammentato in strisce su macchine diverse, il sistema è costretto ad accedere sempre alla rete per raccogliere i singoli chunk e ricomporre il blocco logico.

Goals of a DFS

Big data support

- Requirement: files may be too big to be processed by a single machine (or even bigger than single disks)
- Typical solution: split files into smaller blocks, store blocks on different machines

Abstraction

- Requirement: users should interact with disks on different machines as if they were one, unified disk
- Typical solution: master-slave architecture with high-availability Client communicates only with the Master and the whole system handles the distribution, retrieval of the data

Fault-tolerance

- Requirement: disks can fail, machines can be unreachable, but data should always be available
- Typical solution: store multiple copies of each block (replication)

HDFS: not always the best fit

Although this may change in the future, there are areas where HDFS is not a good fit today

Low-latency data access Accesso ai dati a bassa latenza

- HDFS is optimized for delivering a high throughput of data, and this may be at the expense of latency. Applications that require access to data in the tens of milliseconds range will not work well with HDFS. andare a discapito della

Lots of small files Grande quantità di file piccoli

- The limit to the number of files in a filesystem is determinato dalla governed by the amount of memory RAM on the NN, because it holds filesystem metadata in memory.
- Sebbene Although storing millions of files is feasible, billions is beyond the capability of current hardware.

HDFS: I/O

An application client wishing to read a file (or a portion thereof) must first contact the NN to determine where the actual data is stored

In response to the client request the NN returns:

- The relevant block id
- The location where the block is held (i.e., which DN)

The client then contacts the DN to retrieve the data

Blocks are themselves stored on standard single-machine file systems

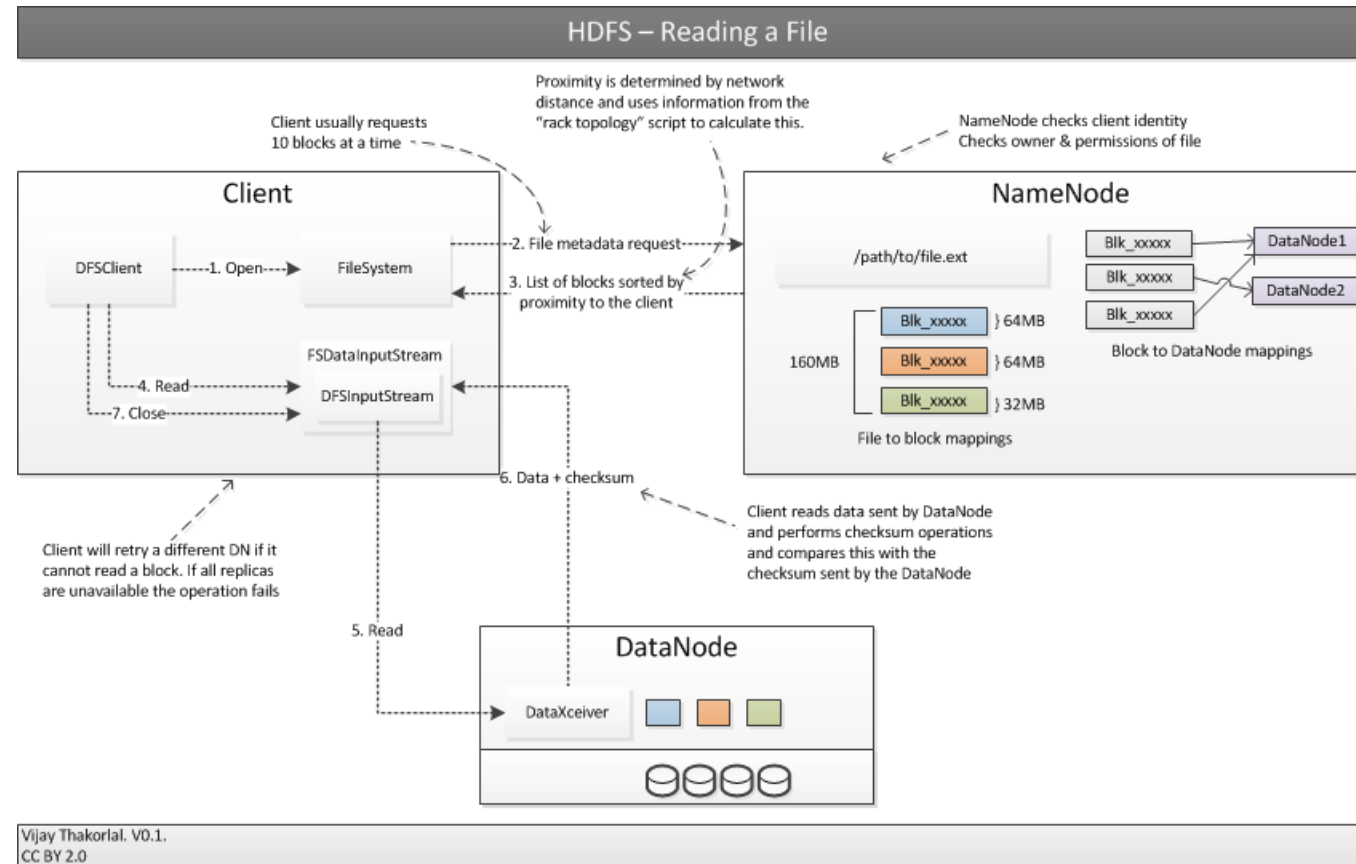
- HDFS lies on top of the standard OS stack

Important features of the design:

- Data is never moved through the NN
- All data transfer occurs directly between clients and DNs
- Communications with the NN only involve transfer of metadata

Reading in HDFS

Only to see the sequence of communication



Only to see the sequence of communication



File formats

Standard formats are not suitable
Row-oriented vs Column-oriented
Apache Parquet for nested structures

File formats

Data usually comes in standard file formats

- Structured CSV files
- Semi-structured JSON files
- Unstructured textual files

However, there are file formats more suitable for big data infrastructures and applications

- Splittable file formats
- Column-oriented (vs row-oriented) file formats

Why using big data formats

Binary serialization

- Compact streams of bytes occupy less space
- Without it, you incur in large overhead for storing data as text
 - More space required
 - Type-conversion required when reading and writing

{ "deposit_money": "12345678" }	
JSON	Binary
'0x6d', '0x6f', '0x6e', '0x65', '0x79', '0x31', '0x32', '0x33', '0x34', '0x35', '0x36', '0x37', '0x38'	'0x01', '0xBC614E'

Splittability

- Metadata headers allow to skip unnecessary I/O
- Without it, you cannot access single portions of files

Compression

- Block-level: distinctly compress the single blocks
- Record-level: distinctly compress the single records within a block
- Otherwise, the whole file must be compressed and decompressed for reading

Row-oriented vs Column-oriented

Disks store data sequentially (they are 1-dimensional)

- Tables cannot be directly represented
- Cells are stored either row-by-row or column-by-column

A	B	C
A1	B1	C1
A2	B2	C2
A3	B3	C3

A1	B1	C1	A2	B2	C2	A3	B3	C3
----	----	----	----	----	----	----	----	----

Row-oriented

A1	A2	A3	B1	B2	B3	C1	C2	C3
----	----	----	----	----	----	----	----	----

Column-oriented

Row orientation is better for transactional workloads (OLTP)

- Applications work on a row basis (e.g., updating a customers' profile, adding a new order)

Column orientation is better for analytical workloads (OLAP)

- Analyses are done on a columnar basis (e.g., the average salary of employees by department)

Row-oriented vs Column-oriented

There are several advantages to column-oriented (columnar) file formats

- Better compression
 - Data is more homogenous
 - The space savings are very noticeable at the scale of a Hadoop cluster
 - Type-specific encodings
- Reduced I/O for analytical queries
 - Only a subset of the columns are read
 - Skip unnecessary deserialization
- Operate on encoded data (e.g., when a dictionary encoding is used)

Row-oriented file formats

Many formats

- **SequenceFiles**
 - Binary key-value pairs
 - Specifically designed to work with MapReduce (but others are available; e.g., MapFiles)
 - No support besides Java
 - Mostly used for Intermediate data storage and to packing many small files into a single SequenceFile
- **Apache Thrift** (Facebook), **Protocol Buffers** (Google)
 - Born to facilitate communication between programs (RPC)
 - Multi-language support
 - Key features are available only through external libraries
- **Apache Avro**
 - Addresses the major drawbacks of Writable (the default serialization of SequenceFiles)
 - Multi-language support
 - More integrated with Hadoop
 - Supports schema evolution, no need to compile code

Row-oriented file formats

Thrift

```
namespace java serializers.thrift.media

typedef i32 int
typedef i64 long

enum Size {
    SMALL = 0,
    LARGE = 1,
}

enum Player {
    JAVA = 0,
    FLASH = 1,
}

struct Image {
    1: string uri, //url to the images
    2: optional string title,
    3: required int width,
    4: required int height,
    5: required Size size,
}

struct Media {
    1: string uri, //url to the thumbnail
    2: optional string title,
    3: required int width,
    4: required int height,
    5: required list<string> person,
    6: required Player player,
    7: optional string copyright,
}

struct MediaContent {
    1: required list<Image> image,
    2: required Media media,
}
```

Protobuf

```
package serializers.protobuf.media;

option java_package = "serializers.protobuf.media";
option java_outer_classname = "MediaContentHolder";
option optimize_for = SPEED; affects the C++ and Java
code generators

message Image {
    required string uri = 1; //url to the thumbnail
    optional string title = 2; //used in the html
    required int32 width = 3; // of the image
    required int32 height = 4; // of the image
    enum Size {
        SMALL = 0;
        LARGE = 1;
    }
    required Size size = 5;
}

message Media {
    required string uri = 1;
    optional string title = 2;
    required int32 width = 3;
    required int32 height = 4;
    repeated string person = 5;
    enum Player {
        JAVA = 0;
        FLASH = 1;
    }
    required Player player = 6;
    optional string copyright = 7;
}

message MediaContent {
    repeated Image image = 1;
    required Media media = 2;
}
```

compile

```
protoc -I=$SRC_DIR
--java_out=$DST_DIR
$SRC_DIR/addressbook.proto
```

obtain class
with getters
and setters

```
com/example/tutorial/
AddressBookProtos.java
```

In Avro, data is always
serialized with its schema

- Each file is self-describing
- Different schemas can be used for serialization and deserialization (Avro will autonomously handle the missing / extra / modified fields)

Column-oriented file formats

Many formats

- **ORC Files** (Optimized Record Columnar Files)
 - First columnar format on Hadoop (initially *RC Files*)
 - No support for schema evolution
 - Originally designed for Hive alone
- **Apache Parquet**
 - General-purpose
 - Supports schema evolution

Parquet

A columnar storage format for efficient querying of **nested structures** in a **flat format**

A little history:

- 2006: the *Dremel* system, using *ColumnIO* file format, goes in production at Google
- 2010: Google publishes a paper about Dremel
- 2013: Apache Parquet 1.0 is released (open source) as a joint effort between Twitter and Cloudera

Parquet ensures interoperability through several frameworks (Spark, MapReduce, etc.) and query engines (Hive, Impala, Pig, Presto, etc.)

Parquet: data model

Root: a group of fields called a **message**

Field types: **group** or **primitive** (e.g., string)

Frequency: **required** (exactly one), **optional** (zero or one), or **repeated** (zero or more)

```
message AddressBook {  
    required string owner;  
    repeated string ownerPhoneNumbers;  
    repeated group contacts {  
        required string name;  
        optional string phoneNumber;  
    }  
}
```

Protocol Buffers and Thrift have a similar (but more complex) schema, which defines also Maps, Lists and Sets

Parquet: data model examples

Schema: List of Strings	Data: ["a", "b", "c", ...]
<pre>message ExampleList { repeated string list; }</pre>	<pre>{ list: "a", list: "b", list: "c", ... }</pre>

Schema: Map of strings to strings	Data: ["AL" => "Alabama", ...]
<pre>message ExampleMap { repeated group map { required string key; optional string value; } }</pre>	<pre>{ map: { key: "AL", value: "Alabama", }, ... }</pre>

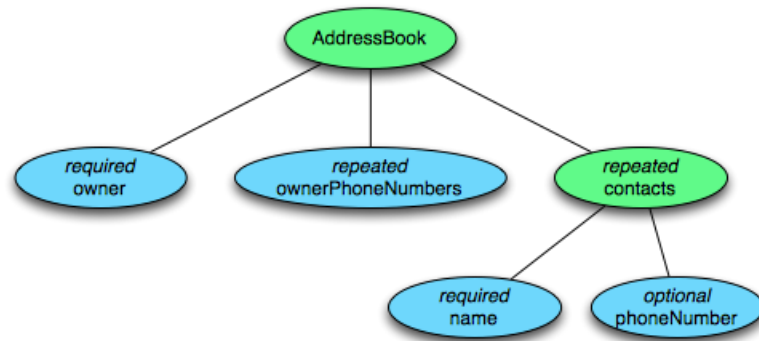
Parquet: unnesting

Problem: can we store nested data structures in columnar format?

- We need to **map the schema to a list of columns** in a way that we can **write records to flat columns** and read them back to their original nested data structure
- **With repeatable fields, how do I know when one record ends and another begins?**

One column per primitive type field in the schema

- If we represent the schema as a tree, the primitive types are the leaves of this tree).



Column	Type
owner	string
ownerPhoneNumbers	string
contacts.name	string
contacts.phoneNumber	string

AddressBook			
owner	ownerPhoneNumbers	contacts	
		name	phoneNumber
...	...	...	...
...	...	...	...
...	...	...	...

Solution

- Each value is associated with two integers: **repetition level** and **definition level**
- These integers allow to **fully reconstruct the nested structures while still being able to store each primitive separately**

Parquet: definition level

The example data contains one column, "**a.b.c**", where all fields are optional and can be *null*.

When **c** is defined, then necessarily **a** and **b** are defined too, but when **c** is *null*, we need to save the level of the *null* value.

- There are 3 nested optional fields, so the maximum definition level is 3.

```
message ExampleDefLevel {  
  optional group a {  
    optional group b {  
      optional string c;  
    }  
  }  
}
```

Value	Definition Level
a: null	0
a: { b: null }	1
a: { b: { c: null } }	2
a: { b: { c: "foo" } }	3 (actually defined)

Parquet: definition level

The maximum definition level is now 2, as **b** does not need one. The value of the definition level for the fields below b changes as follows:

```
message ExampleDefLevel {  
  optional group a {  
    required group b {  
      optional string c;  
    }  
  }  
}
```

Value	Definition Level
a: null	0
a: { b: null }	Impossible, as b is required
a: { b: { c: null } }	1
a: { b: { c: "foo" } }	2 (actually defined)

Remember: the smaller the definition levels, the less the required bits

Parquet: repetition level

The presence of repeated fields requires to store *when* new lists are starting in a column of values. Repetition level indicates **at which level we have to create a new list for the current value.**

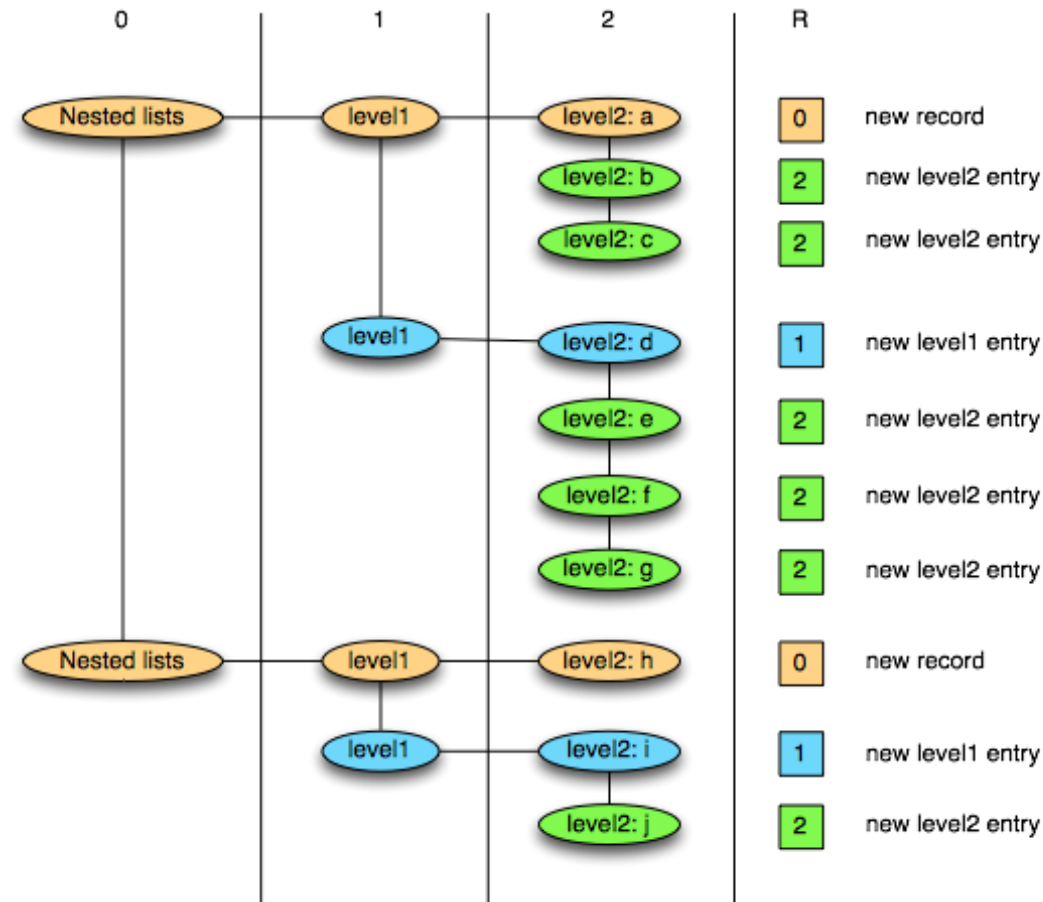
Schema:	Data: [[a,b,c],[d,e,f,g]],[[h],[i,j]]
<pre>message nestedLists { repeated group level1 { repeated string level2; } }</pre>	<pre>{ level1: { level2: a level2: b level2: c }, level1: { level2: d level2: e level2: f level2: g } } { level1: { level2: h }, level1: { level2: i level2: j } }</pre>

The repetition level can be seen as a marker of when to start a new list and at which level.

Repetition level	Value
0	a
2	b
2	c
1	d
2	e
2	f
2	g
0	h
1	i
2	j

- 0 marks every new record;**
implies creating a new level1 and level2 list
- 1 marks every new level1 list;**
implies creating a new level2 list as well
- 2 marks every new element in a level2 list**

Parquet: repetition level



Parquet: AddressBook example

Definition level for column "contacts.phoneNumber"

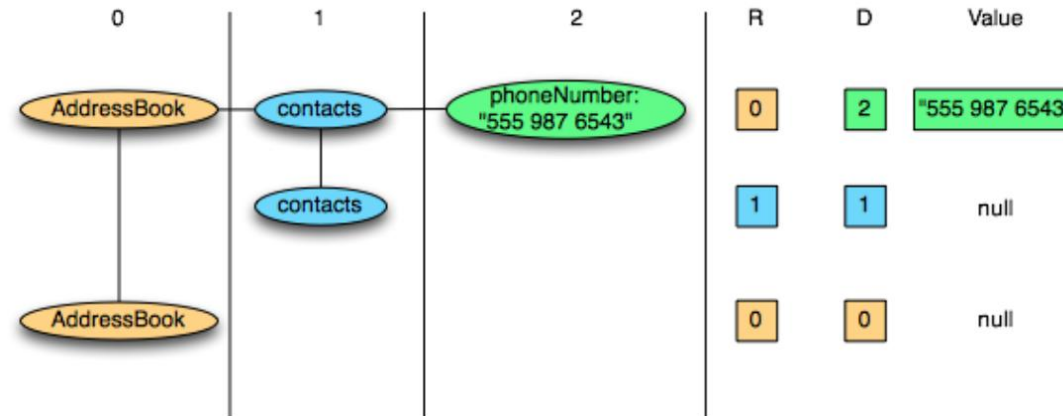
- Defined phone number: 2
- A contact without phone number: 1
- No contacts: 0

Column	Max Definition level	Max Repetition level
owner	0 (owner is <i>required</i>)	0 (no repetition)
ownerPhoneNumbers	1	1 (<i>repeated</i>)
contacts.name	1 (name is <i>required</i>)	1 (contacts is <i>repeated</i>)
contacts.phoneNumber	2 (phoneNumber is <i>optional</i>)	1 (contacts is <i>repeated</i>)

```
AddressBook {  
  owner: "Julien Le Dem",  
  ownerPhoneNumbers: "555 123 4567",  
  ownerPhoneNumbers: "555 666 1337",  
  contacts: {  
    name: "Dmitriy Ryaboy",  
    phoneNumber: "555 987 6543",  
  },  
  contacts: {  
    name: "Chris Aniszczyk"  
  }  
}  
AddressBook {  
  owner: "A. Nonymous"  
}
```

Parquet: AddressBook example

```
AddressBook {  
  contacts: {  
    phoneNumber: "555 987 6543"  
  }  
  contacts: { }  
}  
AddressBook { }
```



Parquet: AddressBook example

To reconstruct the records from the column, we iterate through the column

- **R=0, D=2, Value = “555 987 6543”:**
 - R = 0 means a new record.
We recreate the nested records from the root until the definition level (here 2)
 - D = 2 which is the maximum.
The value is defined and is inserted.
- **R=1, D=1:**
 - R = 1 means a new entry in the contacts list at level 1.
 - D = 1 means contacts is defined but not phoneNumber.
We just create an empty contact.
- **R=0, D=0:**
 - R = 0 means a new record.
We create the nested records from the root until the definition level (here 0)
 - D = 0 => contacts is actually null.
We only have an empty AddressBook.

R	D	Value
0	2	"555 987 6543"
1	1	null
0	0	null

```
AddressBook {  
  contacts: {  
    phoneNumber: "555 987 6543"  
  }  
  contacts: { }  
}  
AddressBook { }
```

Parquet: exercise 1

Define the columns (DL, RL, Value)
for **item** and **quantity**

```
message Person {  
  required string name;  
  
  repeated group orders {  
    required string item;  
    optional int quantity;  
  }  
}
```

```
[  
  {  
    "name": "Alice",  
    "orders": [  
      { "item": "Book", "quantity": 1 },  
      { "item": "Pen", "quantity": null }  
    ]  
  },  
  {  
    "name": "Bob",  
    "orders": null  
  },  
  {  
    "name": "Carol",  
    "orders": [  
      { "item": "Notebook", "quantity": 3 }  
    ]  
  },  
  {  
    "name": "Dan",  
    "orders": []  
  }  
]
```

Parquet: exercise 1 solution

Define the columns (DL, RL, Value)
for **item** and **quantity**

```
message Person {  
  required string name;
```

```
  repeated group orders {  
    required string item;  
    optional int quantity;  
  }  
}
```

DL	RL	item
1	0	Book
1	1	Pen
0	0	Null
1	0	Notebook
0	0	null

```
[  
  {  
    "name": "Alice",  
    "orders": [  
      { "item": "Book", "quantity": 1 },  
      { "item": "Pen", "quantity": null }  
    ]  
  },  
  {  
    "name": "Bob",  
    "orders": null  
  },  
  {  
    "name": "Carol",  
    "orders": [  
      { "item": "Notebook", "quantity": 3 }  
    ]  
  },  
  {  
    "name": "Dan",  
    "orders": []  
  }  
]
```

DL	RL	quantity
2	0	1
1	1	null
0	0	Null
2	0	3
0	0	null

Parquet: exercise 2

Rebuild the messages, starting from the (DL, RL, Value) of **item** and **quantity**

```
message Person {  
  required string name;  
  
  repeated group orders {  
    required string item;  
    optional int quantity;  
  }  
}
```

DL	RL	item
1	0	A
1	1	B
0	0	null
1	0	C
0	0	null
1	0	D
1	1	E
1	1	F
1	1	G

DL	RL	quantity
1	0	null
2	1	101
0	0	null
2	0	102
0	0	null
2	0	103
1	1	null
2	1	104
1	1	null

Parquet: exercise 2 solution

Rebuild the messages, starting from the (DL, RL, Value) of **item** and **quantity**

```
message Person {  
  required string name;  
  
  repeated group orders {  
    required string item;  
    optional int quantity;  
  }  
}
```

```
[{ "orders": [  
  { "item": "A", "quantity": null },  
  { "item": "B", "quantity": 101 }  
]},  
{ "orders": null },  
{ "orders": [  
  { "item": "C", "quantity": 102 }  
]},  
{ "orders": null },  
{ "orders": [  
  { "item": "D", "quantity": 103 }  
  { "item": "E", "quantity": null }  
  { "item": "F", "quantity": 104 }  
  { "item": "G", "quantity": null }  
]}  
]
```

DL	RL	item
1	0	A
1	1	B
0	0	null
1	0	C
0	0	null
1	0	D
1	1	E
1	1	F
1	1	G

DL	RL	quantity
1	0	null
2	1	101
0	0	null
2	0	102
0	0	null
2	0	103
1	1	null
2	1	104
1	1	null

Parquet: storing levels, encodings

Definition and repetition levels cause **overhead**

- Each primitive type correspond three columns
- Their values are bound by the depth of the schema, resulting in **only a few bits used**
- When all fields are *required* in a flat schema, levels can be omitted

Compression takes care of condensing data efficiently

- **Bit Packing**: useful for repetition and definition levels, dictionary keys
- **Run Length Encoding** (RLE): useful for definition level of sparse columns
- **Dictionary Encoding**: useful for columns with few (<50k) distinct values
- Definition of new encodings is supported

Parquet: file format

File: an HDFS file

- Includes the metadata for the file

Row group: a logical horizontal partitioning of the data into rows

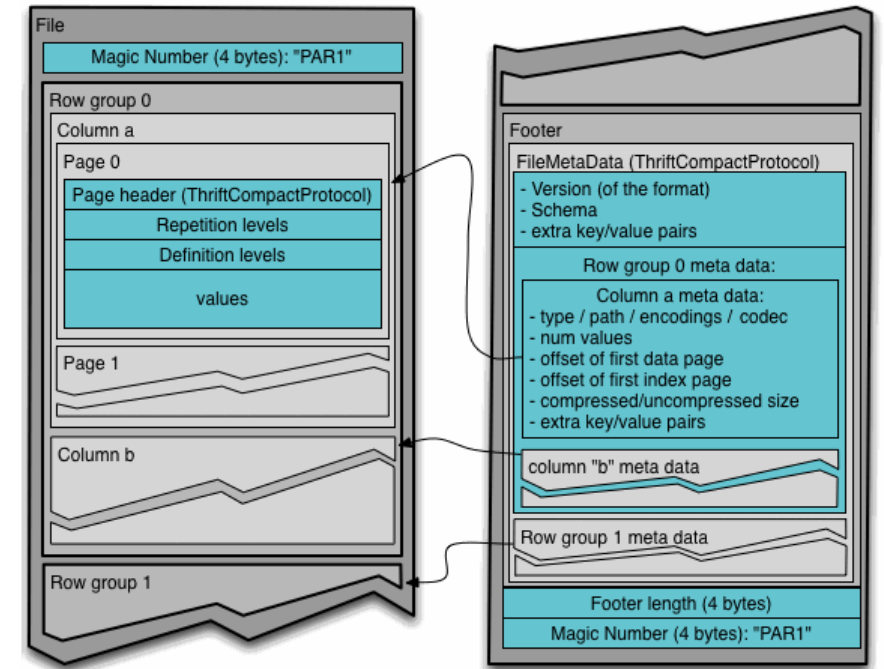
- No physical structure is guaranteed for a row group
- Contains a column chunk for each column in the dataset

Column chunk: a chunk of the data for a particular column

- These live in a particular row group and are guaranteed to be contiguous in the file

Page: column chunks are divided up into pages

- An indivisible unit in terms of compression and encoding



Parquet: configuration

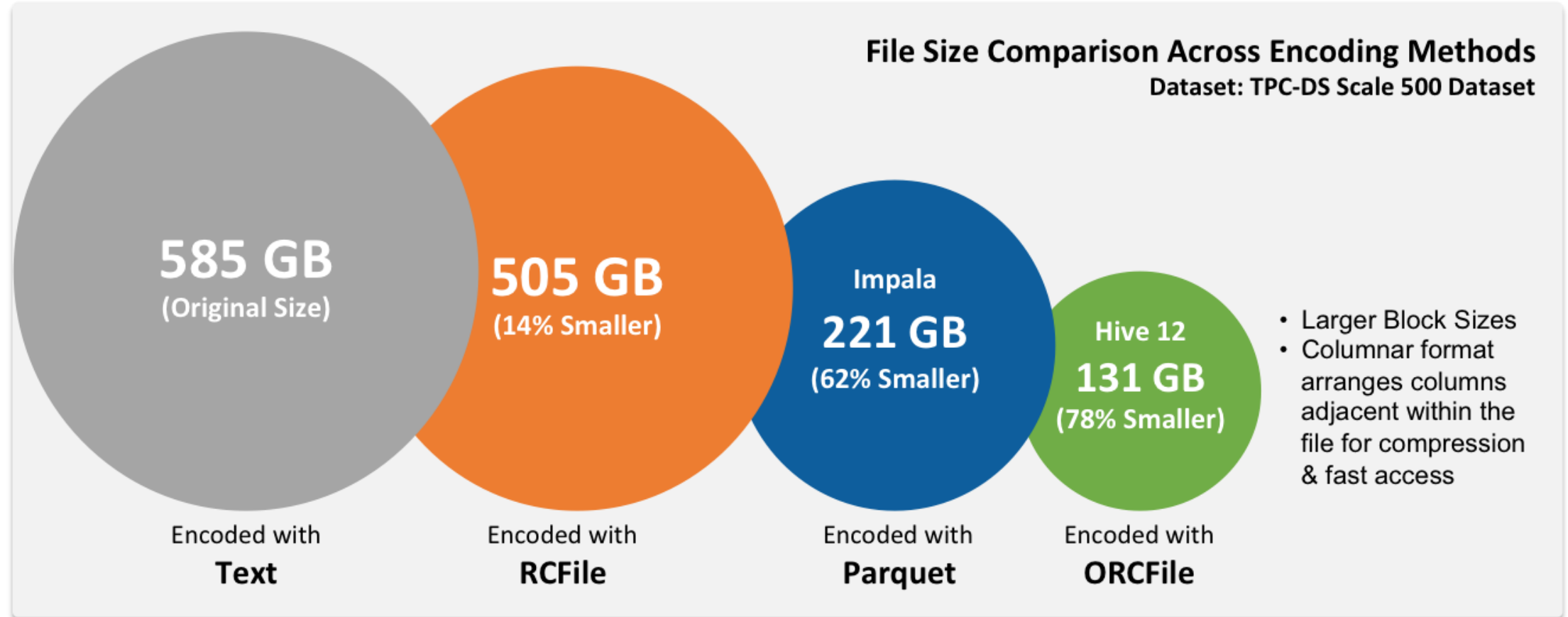
Row group size: larger row groups → larger column chunks → larger sequential IO

- Large row groups (512MB - 1GB) are recommended
 - Since an entire row group might need to be read, we want it to completely fit on one HDFS block
 - Optimized read setup: 1GB row groups, 1GB HDFS block size, 1 HDFS block per HDFS file

Data page size: data pages should be considered indivisible

- Smaller data pages → more fine grained reading (e.g. single row lookup)
- Larger page sizes → less space overhead (less page headers), less parsing overhead (processing headers)

Parquet: efficient storage



Data Lake and Lakehouse

Organizing the data in the DFS

The Data Lake

The Data Lake is “a central repository system for storage, processing, and analysis of raw data, in which the data is **kept in its original format** and is processed to be queried only when needed”

- In the absence of organization and governance, the Data **Lake** can easily become a **Swamp**

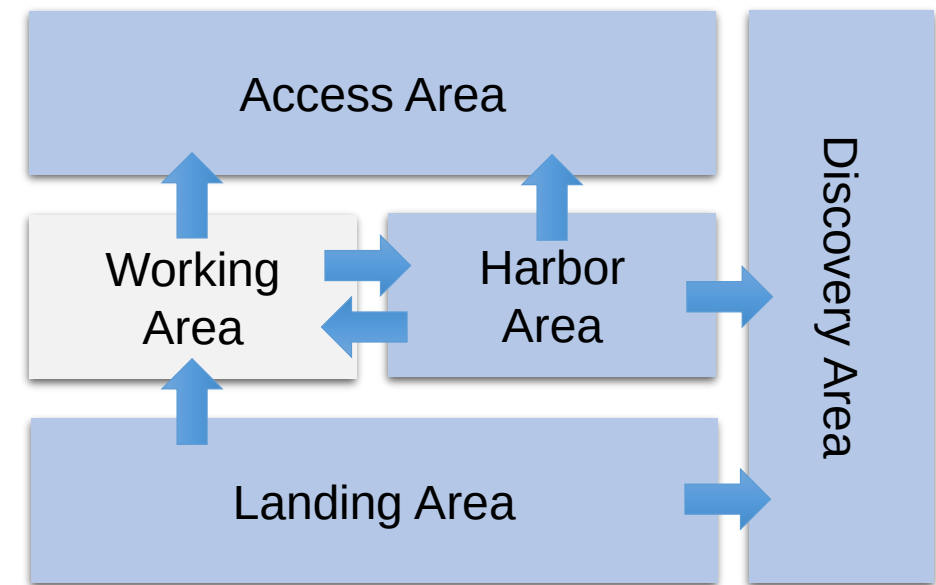


J. Couto, O. T. Borges, D. D. Ruiz, S. Marczak, R. Prikladnicki, *A mapping study about data lakes: An improved definition and possible architectures*, in: Proc. SEKE, Lisbon, Portugal, 2019, pp. 453–578.

The Data Lake

To facilitate data governance, the Data Lake can be partitioned into areas, representing the processing state of the data

- **Landing:** where data are stored immediately after ingestion
- **Working:** it contains temporary tables and configuration data
- **Harbor:** it contains data (completely or partially) processed from the landing area
- **Discovery:** it contains data created by data scientists for experimental purposes; it serves as a safe sandbox, where data is only taken from (and not published to) other areas
- **Access:** it contains data ready to be accessed by reporting and visualization tools

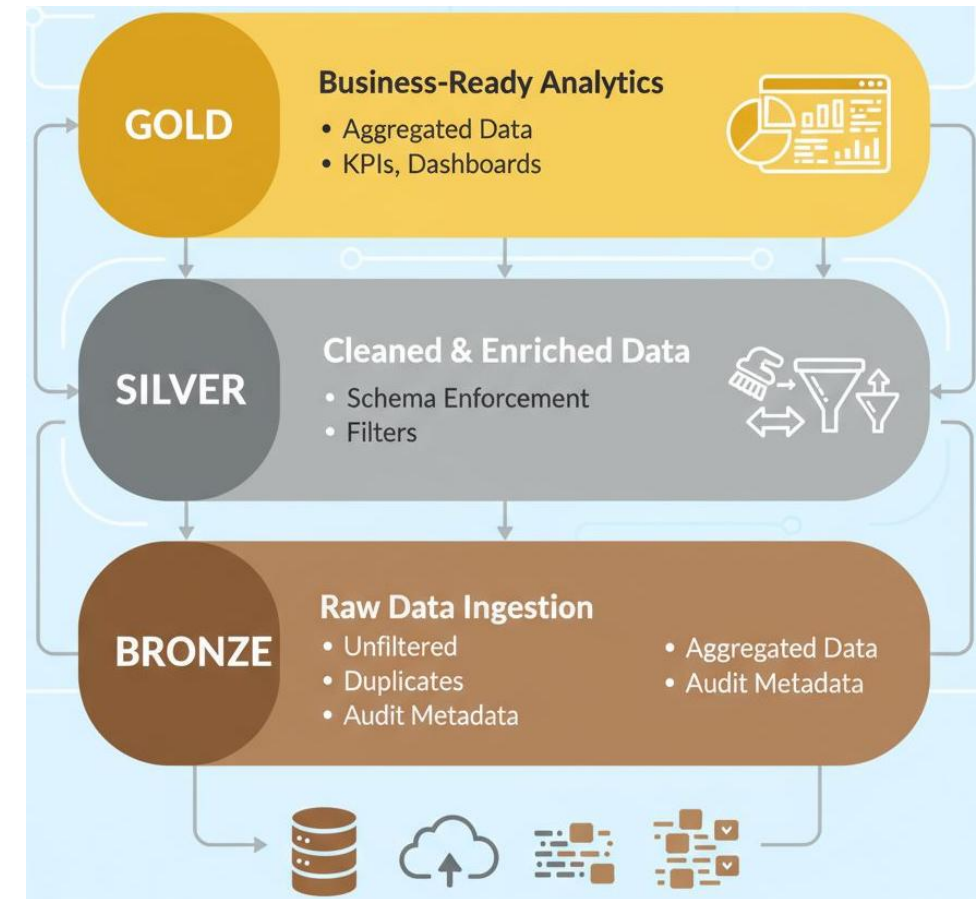


Matteo Francia, Enrico Gallinucci, Matteo Golfarelli, Anna Giulia Leoni, Stefano Rizzi, and Nicola Santolini. "Making data platforms smarter with MOSES." *Future Generation Computer Systems* 125 (2021): 299-313.

The Data Lake

A common reference today is the **medallion architecture**

- **Bronze:** where data are stored immediately after ingestion; a common practice here is to apply only minimal transformations on the data (e.g., type conversions)
- **Silver:** it contains data processed from the bronze layer; it represents the point at which raw data is filtered, cleaned, and “dressed up” (augmented) by joining across one or many other tables
- **Gold:** it contains data ready to be accessed by reporting and visualization tools; it is purpose-built to solve explicit intended business goals



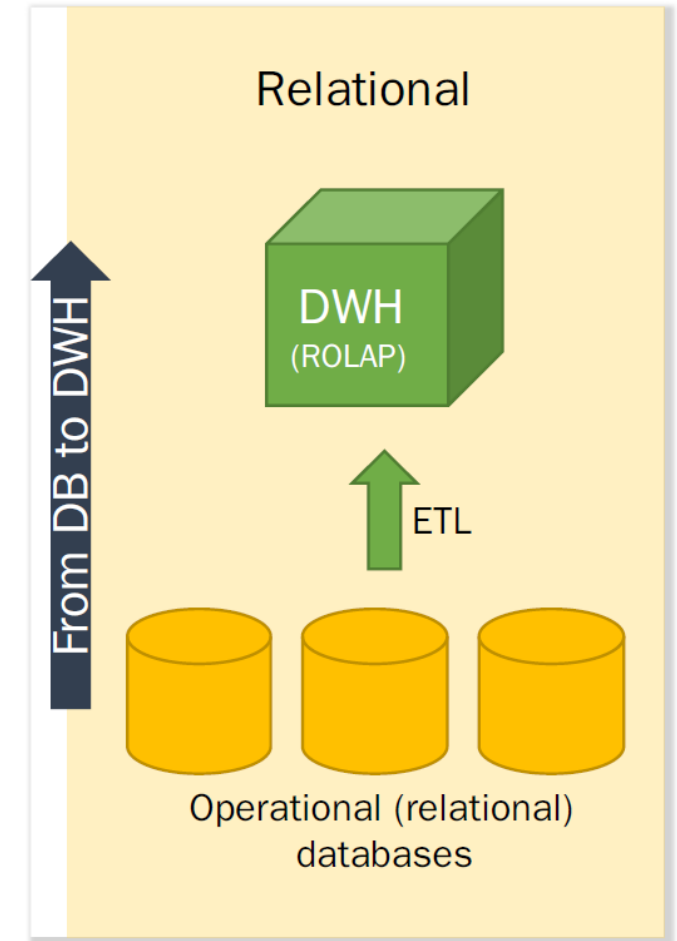
Strengtholt, P., 2025. *Building Medallion Architectures: Designing with Delta Lake and Spark*. " O'Reilly Media, Inc."

Data Warehouse

The transformation from raw data to analytics-ready data is similar to what is traditionally done in Business Intelligence and Data Warehousing

A data warehouse is a collection of data that supports decision-making processes

- It is subject-oriented, integrated and consistent, not volatile
- Typically implemented on top of relational DBMSs (ROLAP)



Golfarelli, M. and Rizzi, S., 2009. Data Warehouse design: Modern principles and Methodologies 5 McGraw-Hill. New York.

The Data Lake

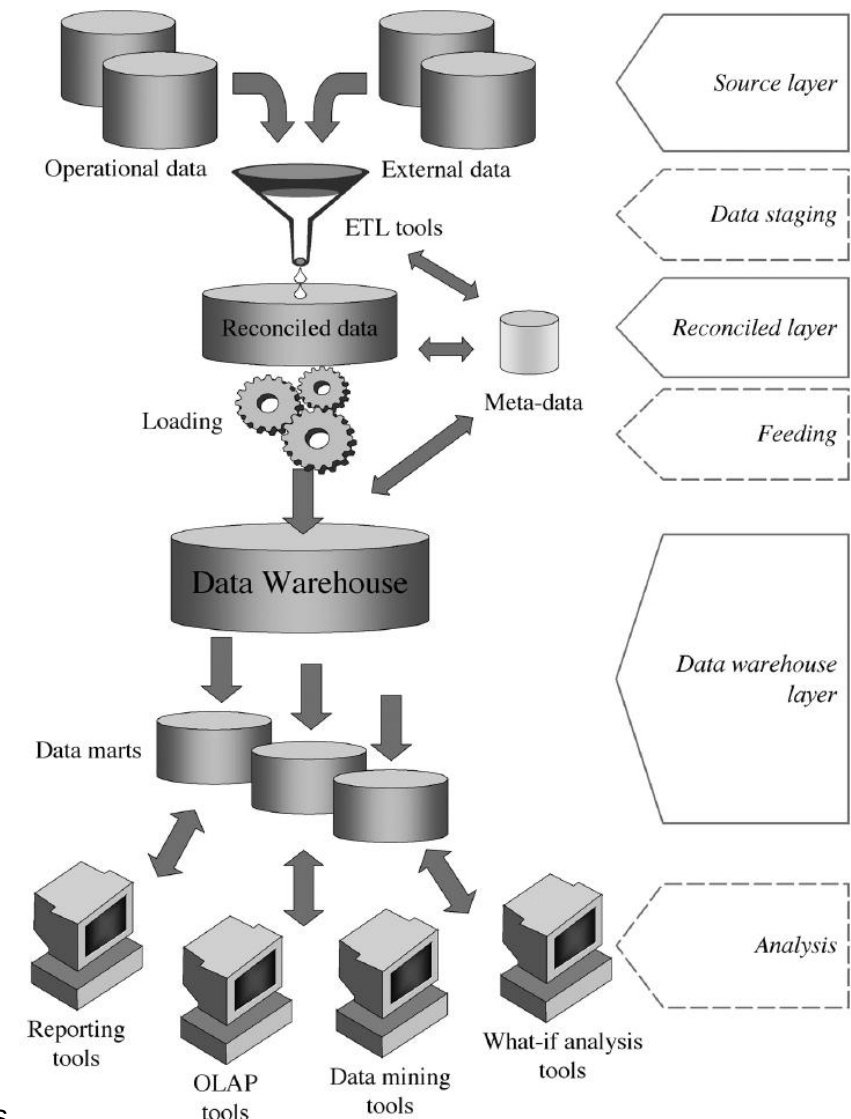
Is the medallion architecture just a “revamped” version of the 3-layer data warehouse architecture?

- Raw area = Bronze level = Source layer
- Harbor area = Silver level = Reconciled layer
- Access area = Gold level = Warehouse layer

Perhaps. Ultimately, there is no universal number of areas/levels/layers

- The basic philosophy is common
- Then, you specialize it based on the complexities within your organization and data pipelines

Golfarelli, M. and Rizzi, S., 2009. Data Warehouse design: Modern principles and Methodologies



Data Lakehouse

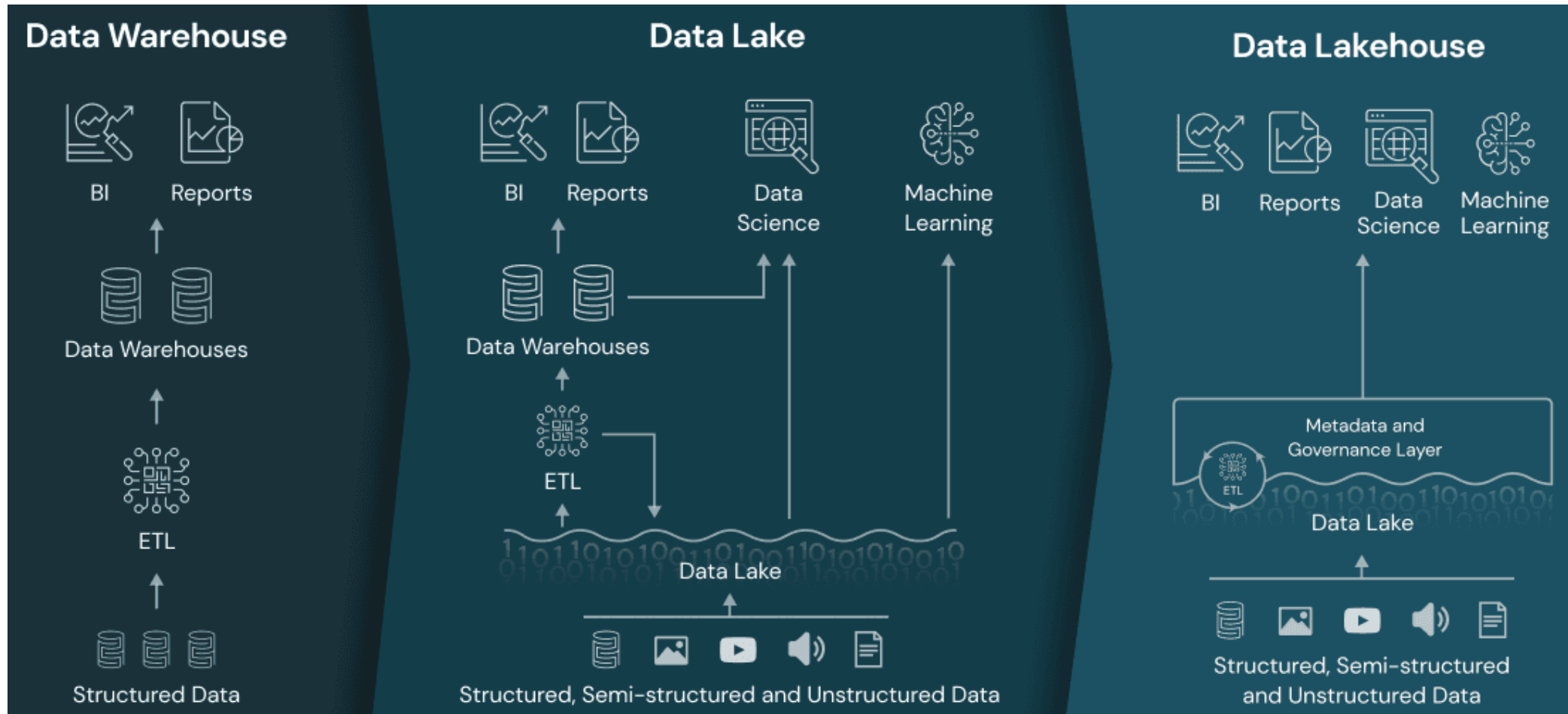
One way or another, raw big data *will be* accessed and processed (transformed, integrated, enriched) for analytical purposes

- You can use a distributed DBMS for DW (e.g., ClickHouse)
- The recent trend is to make the data lake a *lakehouse*

A data lakehouse is a layer that empowers the data lake to become (also) a data warehouse

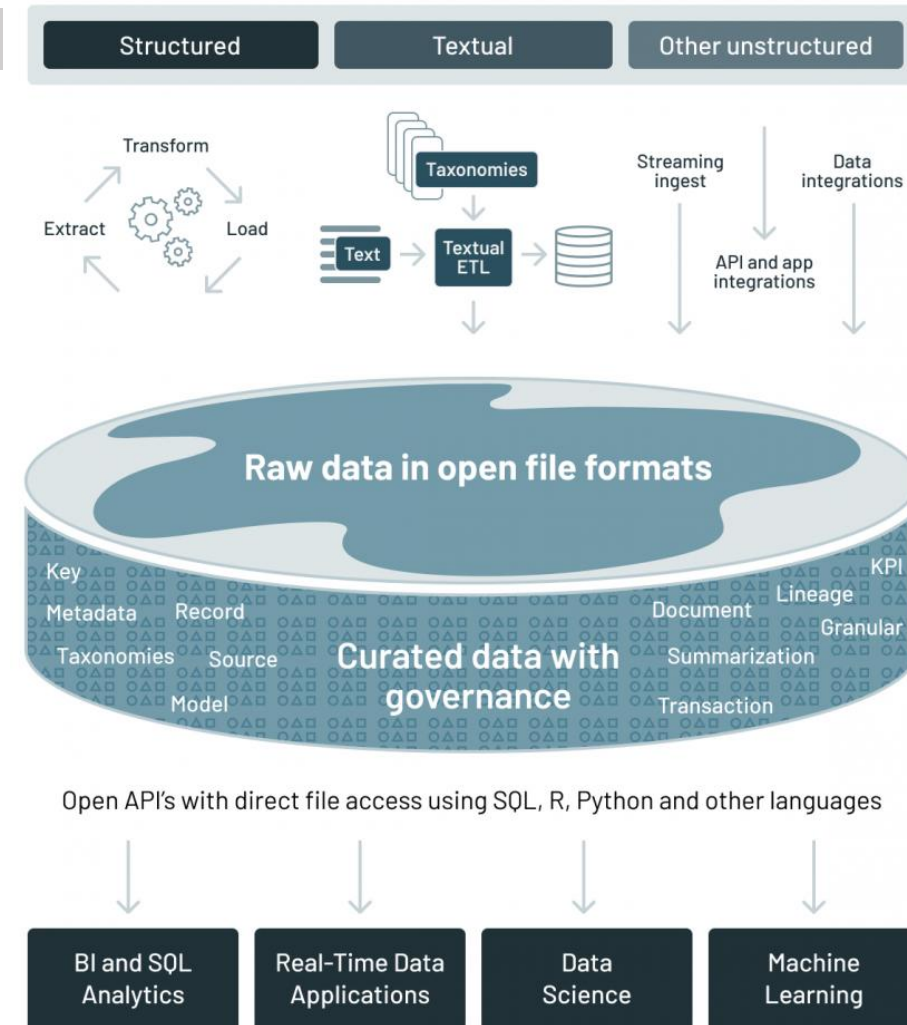
- The goal is to accommodate both the [variety/scalability support of a data lake](#) (which enables data science pipelines) with the [reliability/performance of a data warehouse](#)
- Supports OLTP, but mostly meant for OLAP

Data Lakehouse (the narrative)



Data Lakehouse (the narrative)

	Data warehouse	Data lake	Data lakehouse
Data format	Closed, proprietary format	Open format (e.g., Parquet)	Open format
Types of data	Structured data, with limited support for semi-structured data	All types: Structured data, semi-structured data, textual data, unstructured (raw) data	All types: Structured data, semi-structured data, textual data, unstructured (raw) data
Data access	SQL-only, no direct access to file	Open APIs for direct access to files with SQL, R, Python and other languages	Open APIs for direct access to files with SQL, R, Python and other languages
Reliability	High quality , reliable data with ACID transactions	Low quality, data swamp	High quality, reliable data with ACID transactions
Governance and security	Fine-grained security and governance for row/columnar level for tables	Poor governance as security needs to be applied to files	Fine-grained security and governance for row/columnar level for tables
Performance	High	Low	High
Scalability	Scaling becomes exponentially more expensive	Scales to hold any amount of data at low cost, regardless of type	Scales to hold any amount of data at low cost, regardless of type
Use case support	Limited to BI, SQL applications and decision support	Limited to machine learning	One data architecture for BI, SQL and machine learning



<https://databricks.com/blog/2021/05/19/evolution-to-the-data-lakehouse.html>

Data Lakehouse

How it works

- Data is stored with open, columnar formats (mostly Parquet)
 - Data in other formats (e.g., CSV, JSON) must be converted
- A lakehouse needs a table format
 - A metadata layer ontop of the DFS, managing the underlying file format
 - Allows file to be queried as if they were tables (e.g., with SQL)
 - Common table formats: Delta Lake, Apache Iceberg, Apache Hudi



- The table format provides
 - A catalog that stores metadata about the files
 - A mechanism to track edits and handle conflicts, enabling ACID transactions

Data Lakehouse

The organization of the data in the DFS reflects the structure of a DB

- Based on Apache Hive
- /some-dfs-folder/my-database/
 - ./table-A/
 - ./fileA1.parquet, ./fileA2.parquet, ...
 - ./table-B/
 - ./partition-B1/
 - ./fileB11.parquet, ./fileB12.parquet, ...
 - ./partition-B2/
 - ./fileB11.parquet, ./fileB12.parquet, ...
 - ./table-C/
 - ./bucket-C1/
 - ./fileC11.parquet, ./fileC12.parquet, ...
 - ./bucket-C2/
 - ./fileC11.parquet, ./fileC12.parquet, ...

Based on Apache Hive

Partitioning

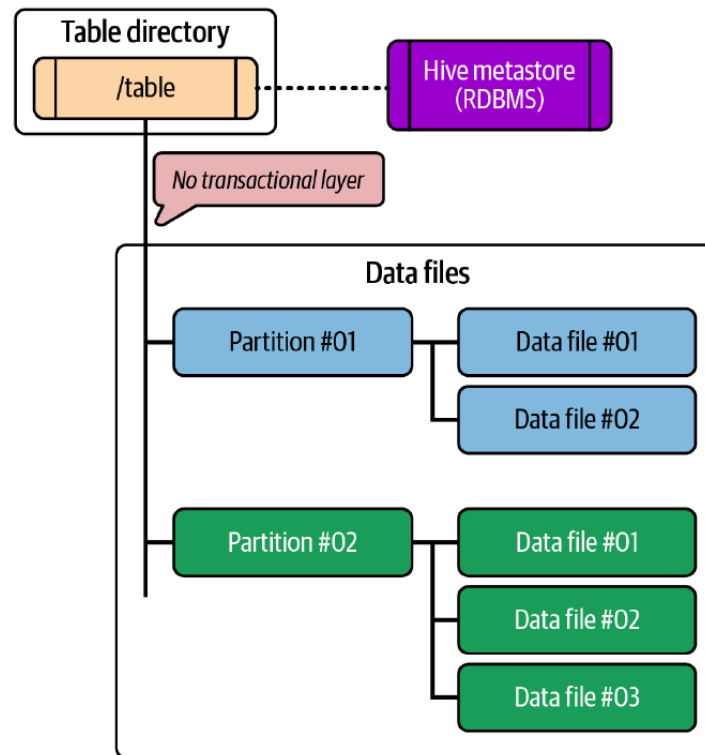
- Stores records based on the value of one (or more) column(s)
- Faster search
- Potential “explosion” of folders
- Ideal for low cardinality columns (e.g., departments)

Bucketing

- Stores records based on the *hash of* a value of one (or more) column(s)
- Faster search (and skipped shuffle)
- The number of buckets is fixed
- Ideal for high cardinality columns (e.g., IDs)

Data Lakehouse

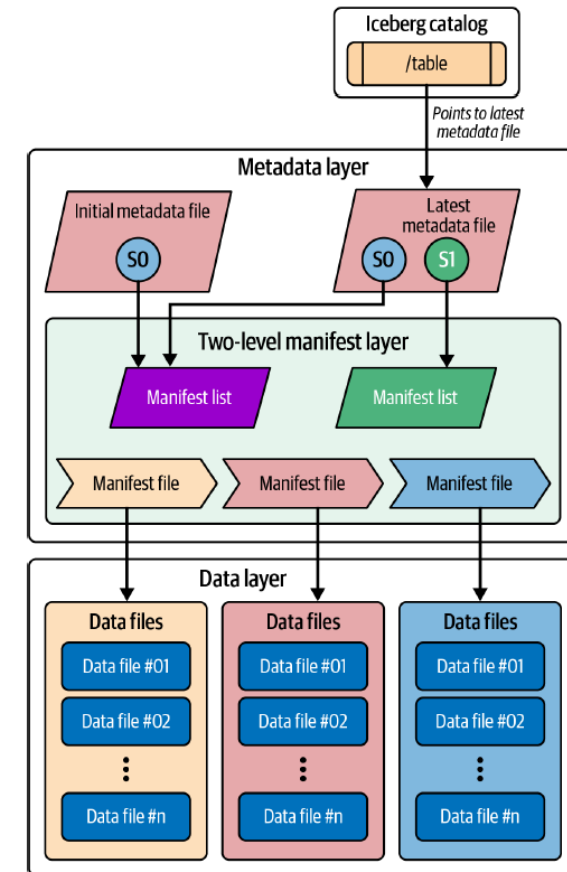
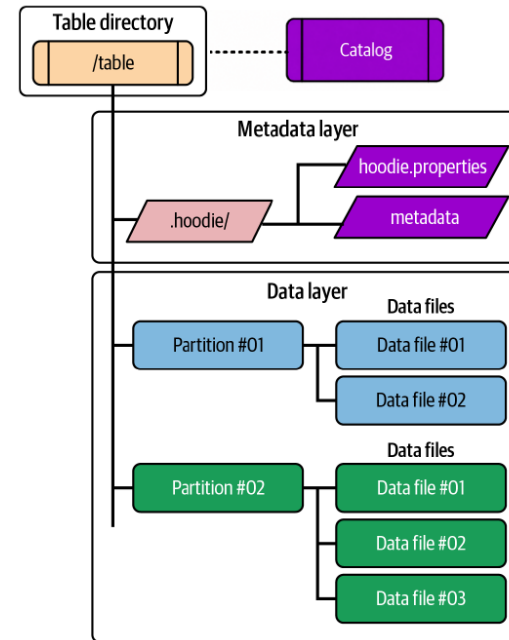
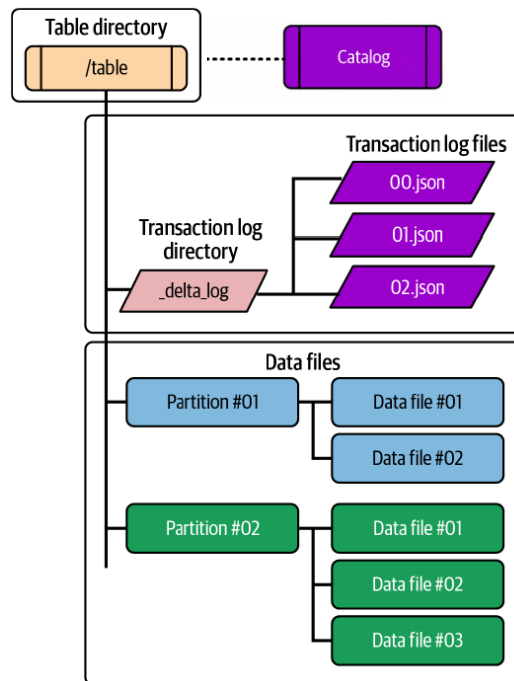
Architecture of Apache Hive



Thalpati, Gaurav Ashok. *Practical Lakehouse Architecture: Designing and Implementing Modern Data Platforms at Scale*. " O'Reilly Media, Inc.", 2024.

Data Lakehouse

Architectures of Delta, Hudi, and Iceberg

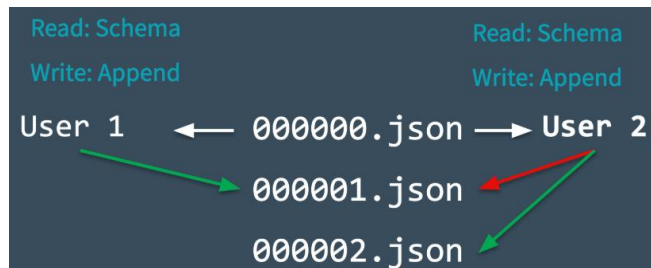


Thalpati, Gaurav Ashok. *Practical Lakehouse Architecture: Designing and Implementing Modern Data Platforms at Scale*. " O'Reilly Media, Inc.", 2024.

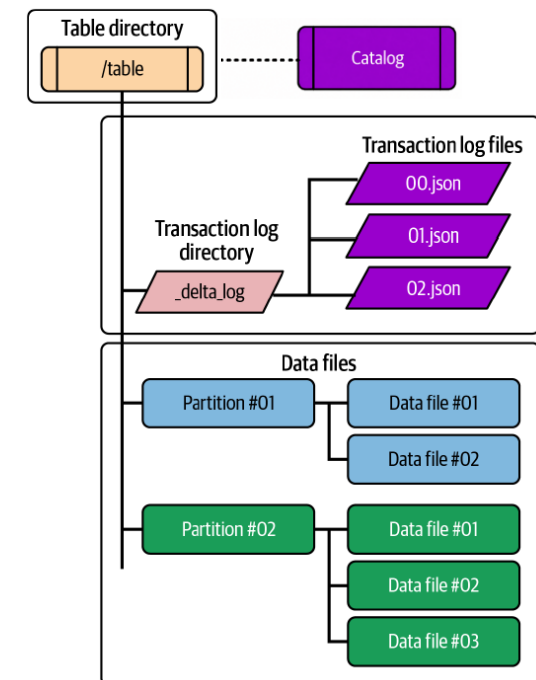
Data Lakehouse

How it works

- In a DFS, files are read-only – so how is data updated?
 - Files are automatically enriched with **metadata files**, logging all edits
- An *optimistic* concurrency approach is adopted
 - Works on the assumption that conflicts are rare
 - Instead of locking data preemptively, conflicts are checked only at the time of the transaction's commit
 - If a conflict is detected, the system takes corrective actions (e.g., aborting or retrying)



- Works well only with few write transactions per second

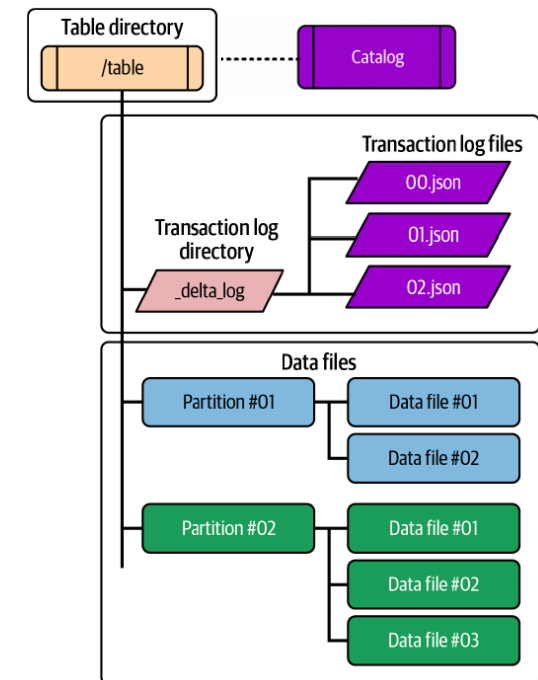
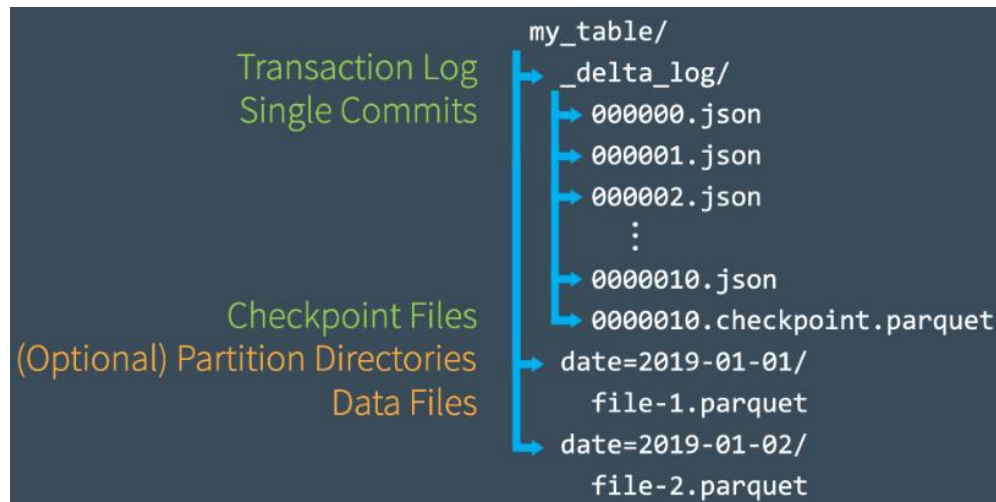


<https://www.databricks.com/blog/2019/08/21/diving-into-delta-lake-unpacking-the-transaction-log.html>

Data Lakehouse

How it works

- The transaction log scale doesn't scale
 - Checkpointing: materialize the new Parquet file resulting from the application of all edits to the previous file



<https://www.databricks.com/blog/2019/08/21/diving-into-delta-lake-unpacking-the-transaction-log.html>

Data Lakehouse

How it works

- How is data accessed?
 - The catalog stores metadata about all the files and enables access to files as if they were tables
 - A separate computation engine must be used to access the files
 - The computation engine exploits the lakehouse format's libraries to
 - Update the data with ACID properties
 - Carry out time-travel, i.e., seeing previous versions of the data, up until transaction logs and checkpoints are cleaned (e.g. 30 days)

